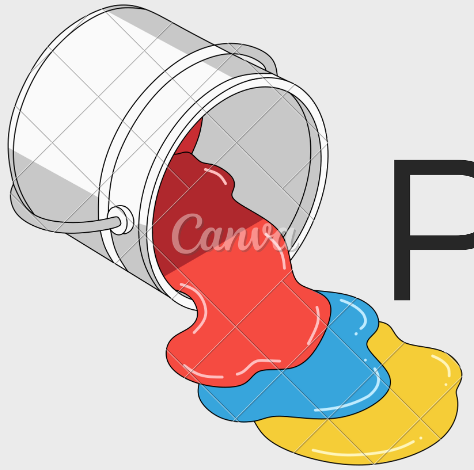
RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT (IRM)

End-to-end data pipeline for inventory obsolescence detection

by Group 6 :
TAN ZHI MING
SAFIYA NURSYAHADAH BINTI MASNOOR
FARRA NURZAHIN BINTI ZAHARIL ANUAR
AIN NURNABILA BINTI MOHD AZHAR
WOO CHENG SHUAN
WUCHENXI



Project Introduction & Purpose

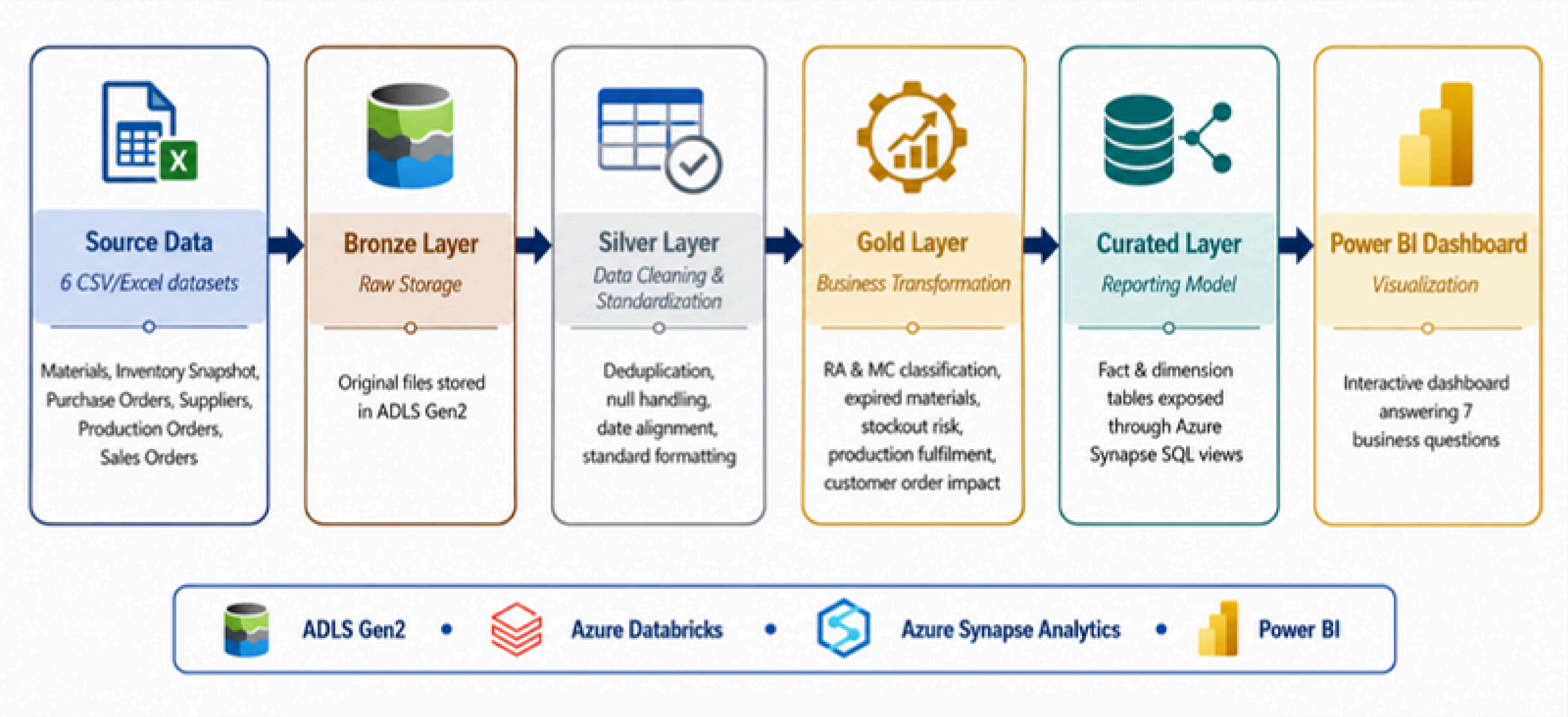
The Balancing Act

Paint manufacturing demands complex raw material management. Holding too much stock ties up capital and risks expiration; too little stock stalls production and delays critical customer orders.

Automated Monitoring

We replace manual spreadsheet tracking with a disciplined, reproducible 4-layer medallion architecture on Microsoft Azure. It continuously detects surplus, expiration, and stockout threats.

Pipeline Architecture



data source

Materials – material master information

Inventory Snapshot – current stock quantity and inventory status

Purchase Orders – supplier replenishment and incoming stock

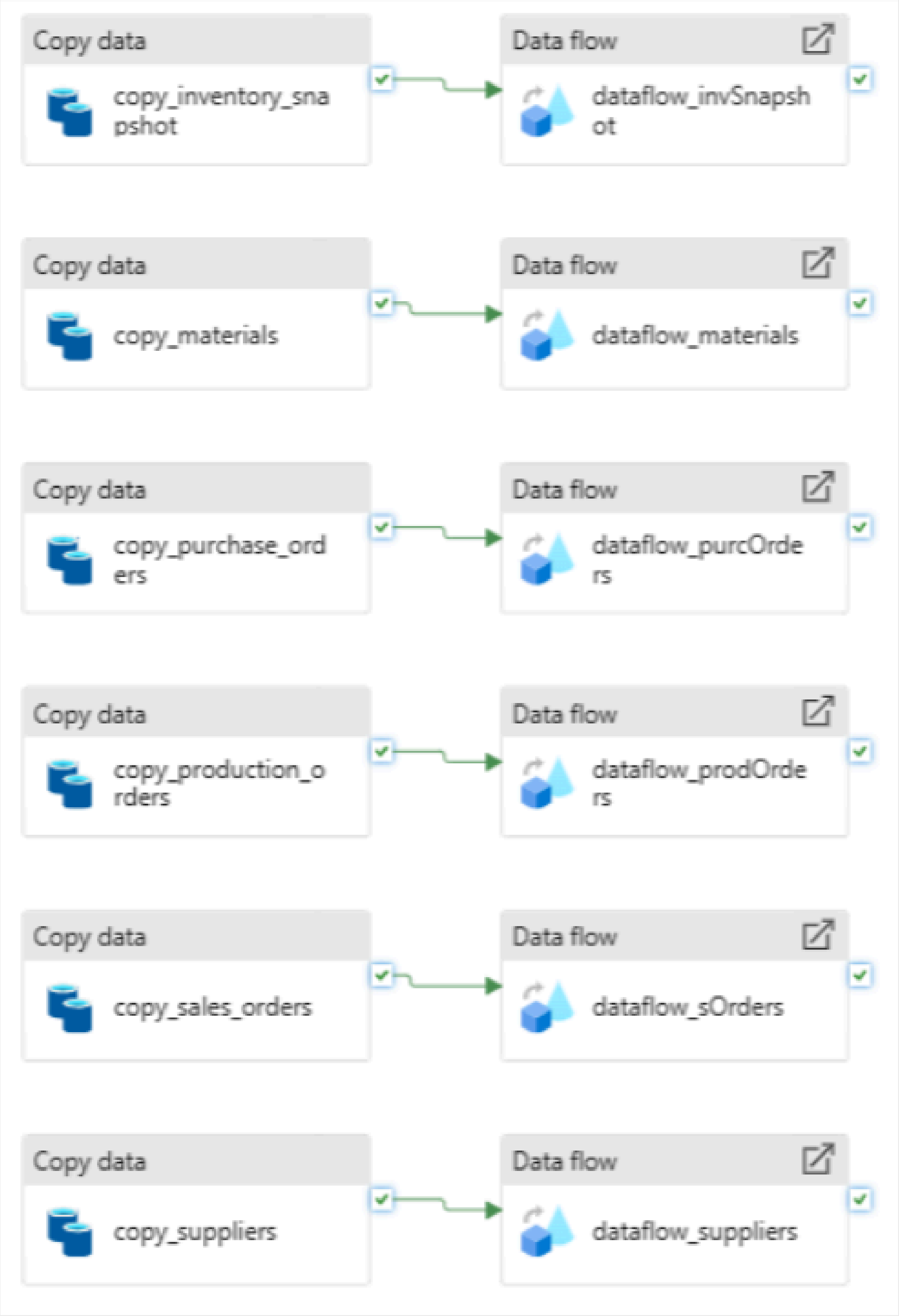
Suppliers – supplier-related information

Production Orders – production demand and fulfilment checking

Sales Orders – customer order impact analysis

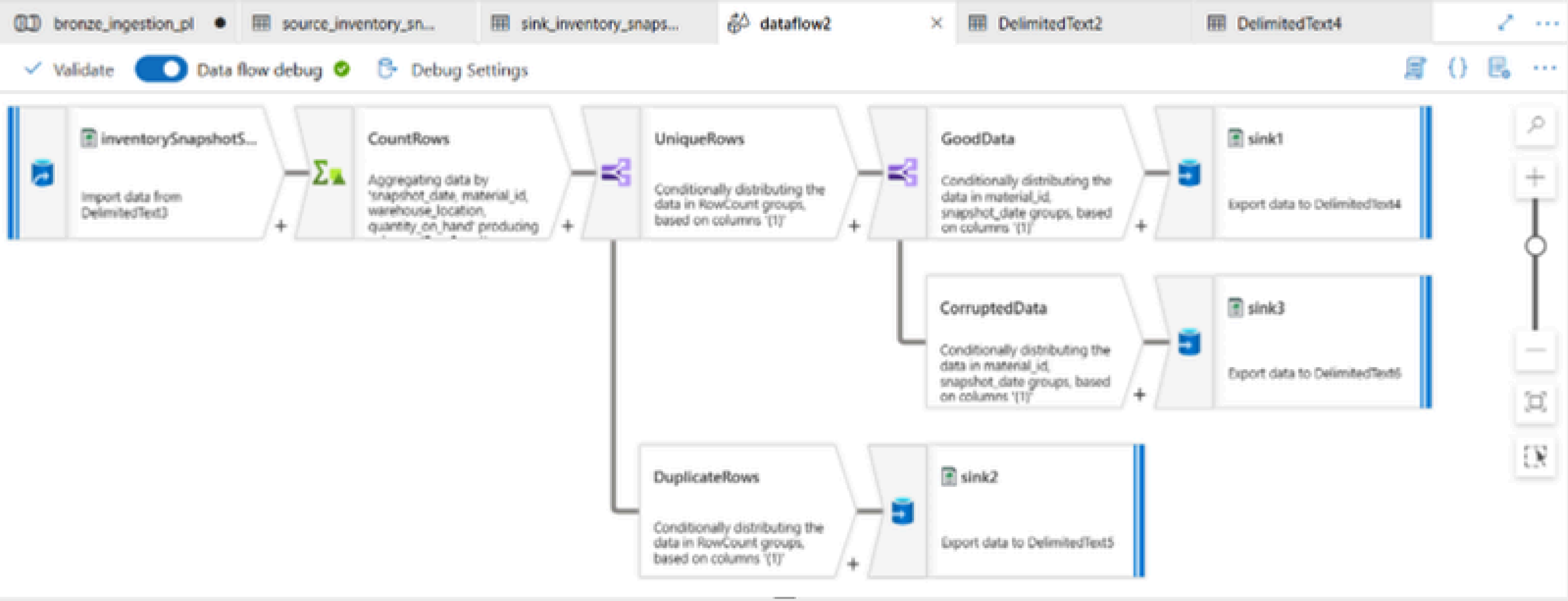
Bronze Layer

Data ingestion & Quality Check



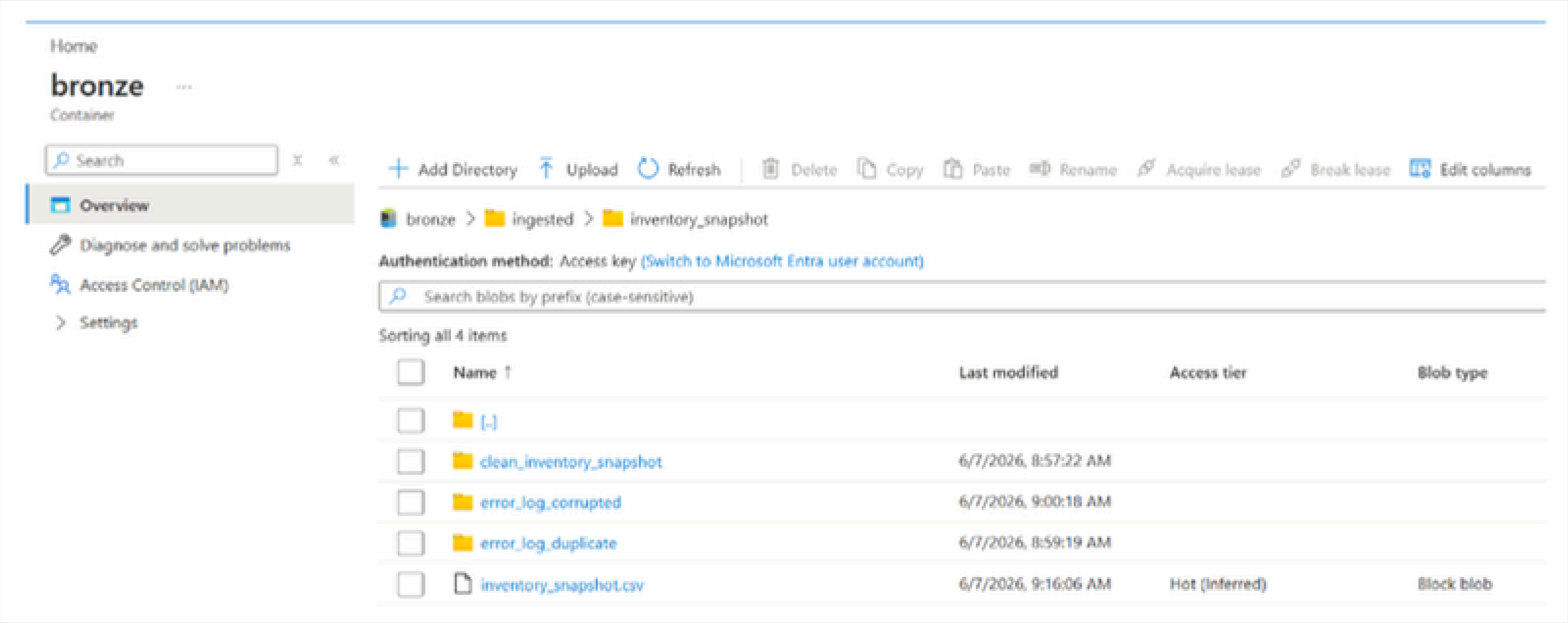
Bronze Layer

Data Quality Check



Bronze Layer

Data Quality Check Output



Silver Layer

Deduplication

18 hours ago (<1s) 4

```python
from pyspark.sql.functions import col, to_date, upper, trim

# Remove duplicates from all 6 datasets
df_materials = df_materials.dropDuplicates()
df_inventory = df_inventory.dropDuplicates()
df_purchase = df_purchase.dropDuplicates()
df_production = df_production.dropDuplicates()
df_suppliers = df_suppliers.dropDuplicates()
df_sales = df_sales.dropDuplicates()

print("Duplicates removed")
```

Null Handling

18 hours ago (1s) 5

```python
# Fill nulls with appropriate defaults
# Materials
df_materials = df_materials.fillna({
    "category": "UNKNOWN",
    "unit_cost": 0.0,
    "reorder_level": 0
})

# Inventory snapshot
df_inventory = df_inventory.fillna({
    "quantity_on_hand": 0
})

# Purchase orders
df_purchase = df_purchase.fillna({
    "quantity_received": 0
})
```

18 hours ago (1s) 5

```python
# Production orders
df_production = df_production.fillna({
    "quantity_consumed": 0
})

# Suppliers
df_suppliers = df_suppliers.fillna({
    "reliability_rating": 0.0,
    "country": "UNKNOWN"
})

# Sales orders
df_sales = df_sales.fillna({
    "quantity_ordered": 0,
    "status": "UNKNOWN"
})

print("Null values handled")
```

Silver Layer

Dates Alignment

18 hours ago (1s) 6

```python
# Fix date columns to proper date format
df_inventory = df_inventory.withColumn(
    "snapshot_date", to_date(col("snapshot_date"), "yyyy-MM-dd")
)

df_purchase = df_purchase.withColumn(
    "po_date", to_date(col("po_date"), "yyyy-MM-dd")
).withColumn(
    "expected_delivery_date", to_date(col("expected_delivery_date"), "yyyy-MM-dd")
).withColumn(
    "actual_delivery_date", to_date(col("actual_delivery_date"), "yyyy-MM-dd")
)

df_production = df_production.withColumn(
    "production_date", to_date(col("production_date"), "yyyy-MM-dd")
)

df_sales = df_sales.withColumn(
    "order_date", to_date(col("order_date"), "yyyy-MM-dd")
).withColumn(
    "required_delivery_date", to_date(col("required_delivery_date"), "yyyy-MM-dd")
)

print("Data types fixed")
```

Standard Formatting

18 hours ago (1s)

```python
# Standardise string casing
df_materials = df_materials.withColumn(
    "category", upper(trim(col("category")))
).withColumn(
    "material_name", upper(trim(col("material_name")))
)

df_suppliers = df_suppliers.withColumn(
    "country", upper(trim(col("country")))
).withColumn(
    "supplier_name", upper(trim(col("supplier_name")))
)

df_sales = df_sales.withColumn(
    "status", upper(trim(col("status")))
).withColumn(
    "customer_name", upper(trim(col("customer_name")))
)

print("Formats standardised")
```

Gold Layer

Business Transformation

May 30, 2026 (2s)

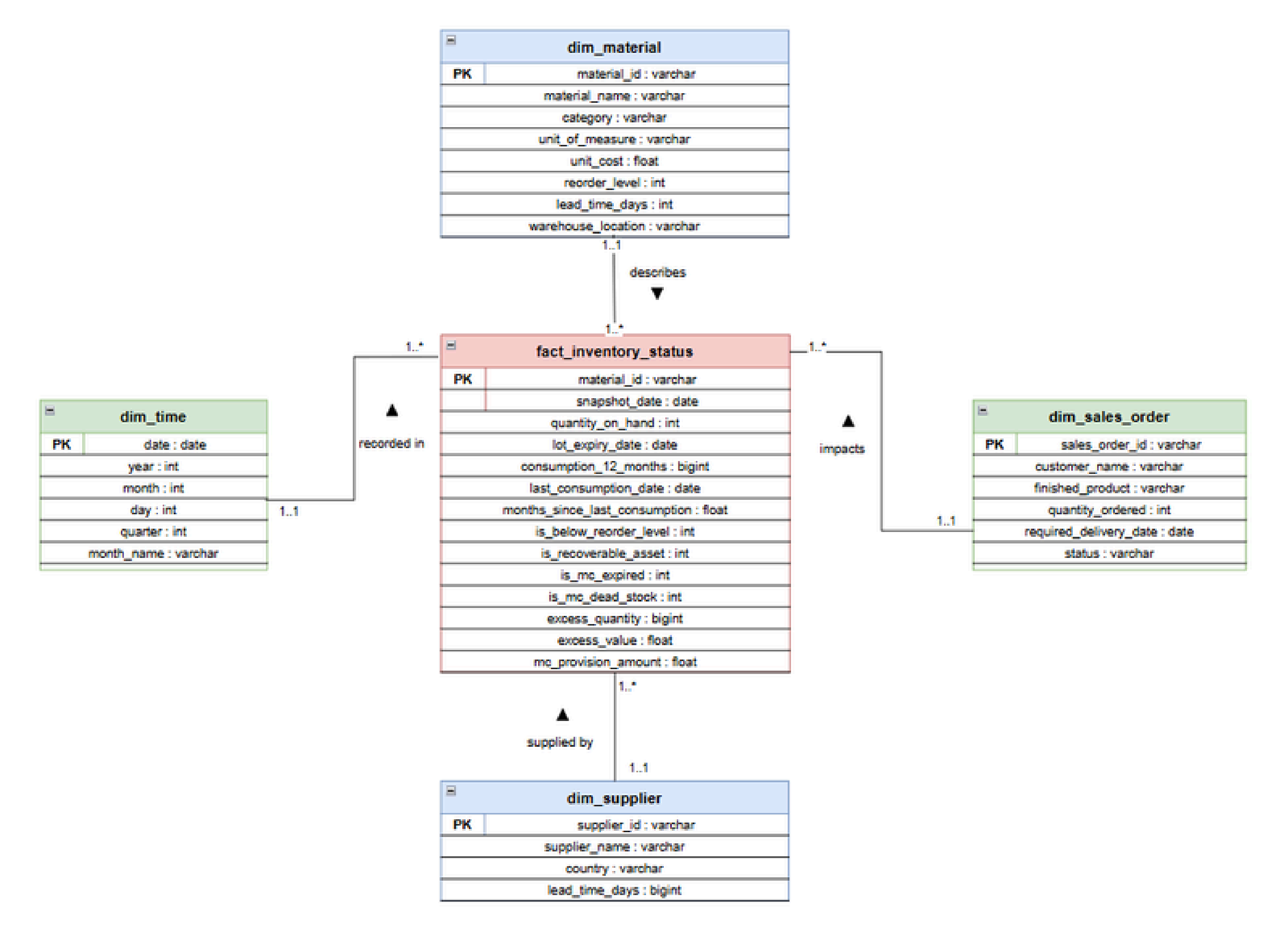
22

```python
gold_stockout_risk = (
    gold_inventory_status
    .filter(F.col("is_below_reorder_level") == 1)
    .withColumn(
        "shortage_quantity",
        F.col("reorder_level") - F.col("quantity_on_hand")
    )
    .select(
        "material_id",
        "material_name",
        "category",
        "quantity_on_hand",
        "reorder_level",
        "shortage_quantity",
        "unit_cost"
    )
)

display(gold_stockout_risk)

gold_stockout_risk.write.mode("overwrite").parquet(
    gold_path + "stockout_risk/"
)
```

(6) Spark Jobs

```python
gold_inventory_status = (
    gold_inventory_status
    .fillna({
        "consumption_12_months": 0,
        "quantity_on_hand": 0,
        "unit_cost": 0,
        "reorder_level": 0
    })
    .withColumn(
        "is_below_reorder_level",
        F.when(F.col("quantity_on_hand") < F.col("reorder_level"), 1).otherwise(0)
    )
    .withColumn(
        "is_recoverable_asset",
        F.when(F.col("quantity_on_hand") > F.col("consumption_12_months"), 1).otherwise(0)
    )
    .withColumn(
        "is_mc_expired",
        F.when(
            F.col("lot_expiry_date").isNotNull() &
            (F.col("lot_expiry_date") < F.current_date()),
            1
        ).otherwise(0)
    )
    .withColumn(
        "is_mc_dead_stock",
        F.when(
            (F.col("months_since_last_consumption") >= 9) &
```

CuratedLayer



Jun 01, 2026 (1s) 6

```python
# CELL 6 — Build Fact Table
fact_inventory_status = df_inventory_status.select(
    col("material_id"),
    col("snapshot_date"),
    col("quantity_on_hand"),
    col("lot_expiry_date"),
    col("consumption_12_months"),
    col("last_consumption_date"),
    col("months_since_last_consumption"),
    col("is_below_reorder_level"),
    col("is_recoverable_asset"),
    col("is_mc_expired"),
    col("is_mc_dead_stock"),
    col("excess_quantity"),
    col("excess_value"),
    col("mc_provision_amount")
)

print("fact_inventory_status:", fact_inventory_status.count())
fact_inventory_status.show(3)
```

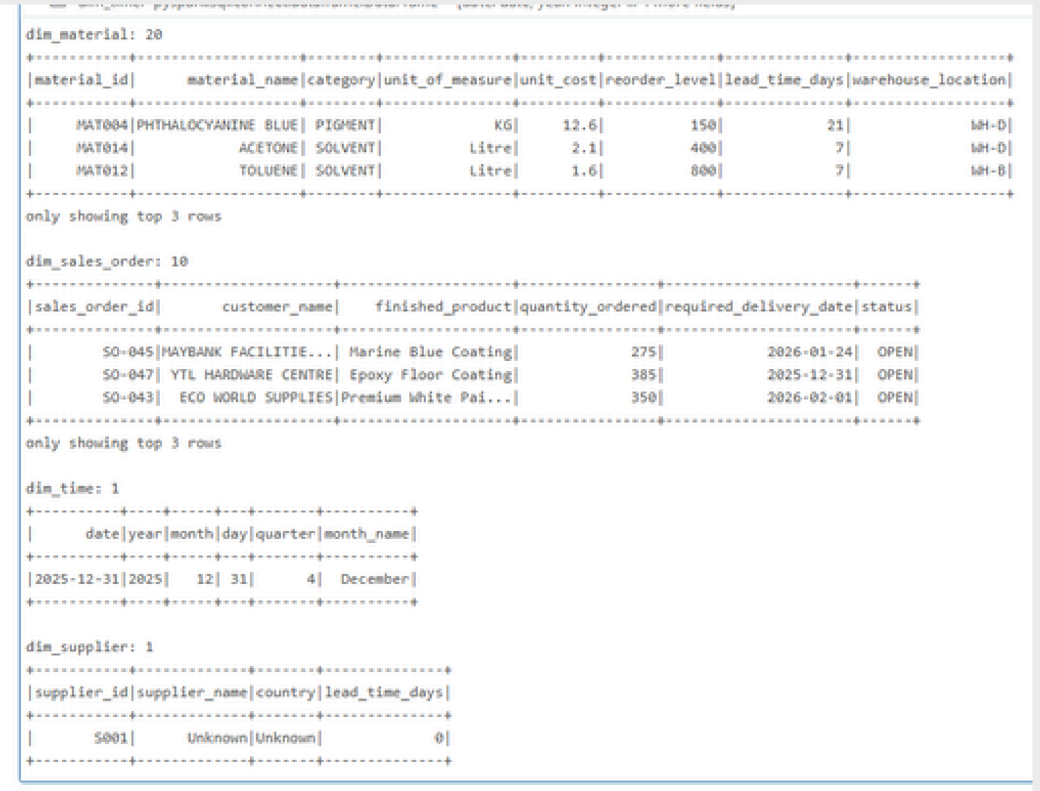
(3) Spark Jobs

fact_inventory_status: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, snapshot_date: date ... 12 more fields]

fact_inventory_status: 20

material_id	snapshot_date	quantity_on_hand	lot_expiry_date	consumption_12_months	last_consumption_date	months_since_last_consumption	is_below_reorder_level	is_recoverable_asset	is_mc_expired	is_mc_dead_stock	excess_quantity	excess_value	mc_provision_amount
MAT001	2025-12-31	380	2026-12-31	7620	2025-12-05	5.80645161	1	0	0	0	0	0.0	0.0
MAT002	2025-12-31	0	NULL	2510	2025-11-12	6.58064516	1	0	0	0	0	0.0	0.0
MAT003	2025-12-31	0	NULL	1960	2025-11-12	6.58064516	1	0	0	0	0	0.0	0.0

only showing top 3 rows



dim_material: 20

material_id	material_name	category	unit_of_measure	unit_cost	reorder_level	lead_time_days	warehouse_location
MAT004	PHTHALOCYANINE BLUE	PIGMENT	KG	12.6	150	21	WH-D
MAT014	ACETONE	SOLVENT	Litre	2.1	400	7	WH-D
MAT012	TOLUENE	SOLVENT	Litre	1.6	800	7	WH-B

only showing top 3 rows

dim_sales_order: 10

sales_order_id	customer_name	finished_product	quantity_ordered	required_delivery_date	status
SO-045	MAYBANK FACILITIE...	Marine Blue Coating	275	2026-01-24	OPEN
SO-047	YTL HARDWARE CENTRE	Epoxy Floor Coating	385	2025-12-31	OPEN
SO-043	ECO WORLD SUPPLIES	Premium White Pai...	350	2026-02-01	OPEN

only showing top 3 rows

dim_time: 1

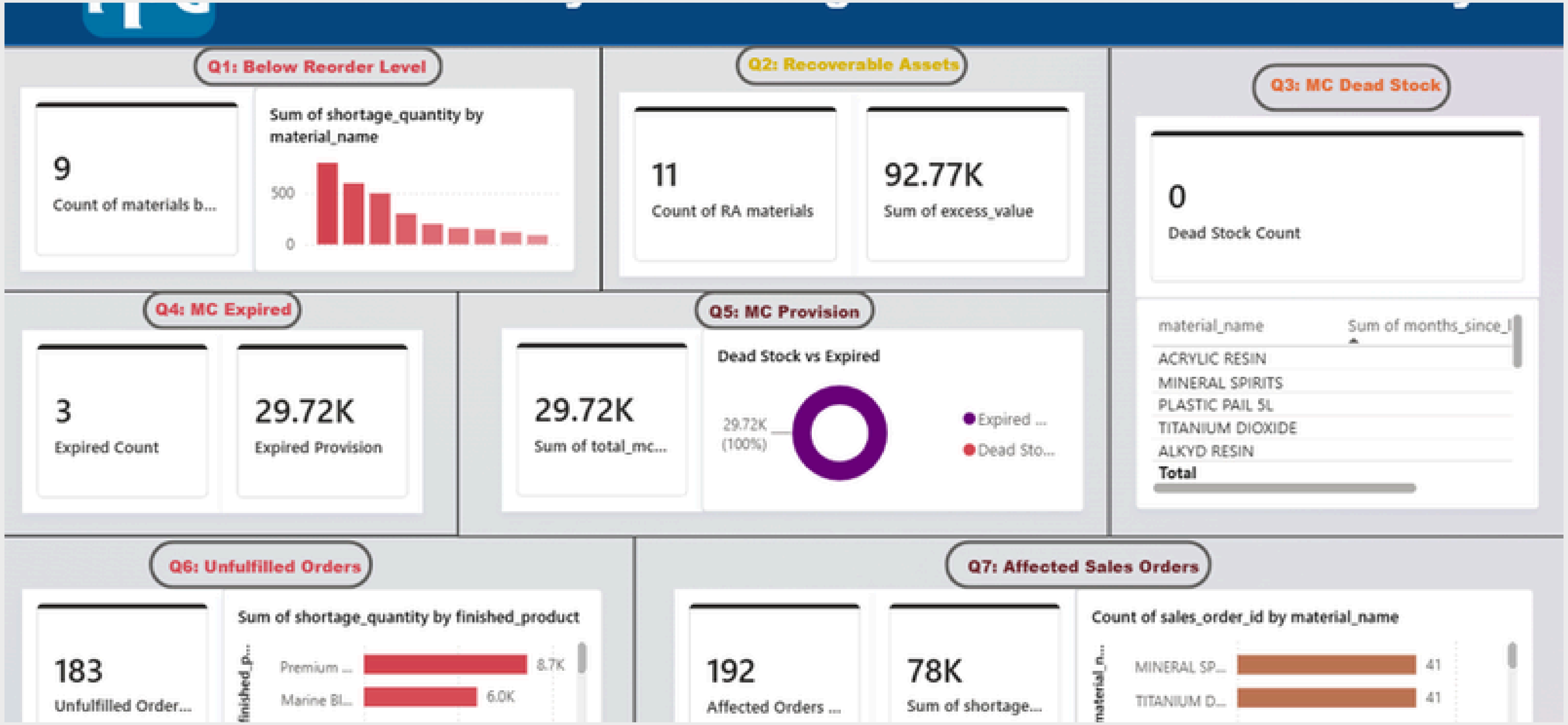
date	year	month	day	quarter	month_name
2025-12-31	2025	12	31	4	December

dim_supplier: 1

supplier_id	supplier_name	country	lead_time_days
S001	Unknown	Unknown	0

- Designed a star schema with 1 fact table (fact_inventory_status) + 4 dimension tables (dim_material, dim_time, dim_sales_order, dim_supplier)
- Loaded Gold data into Databricks, built the schema using PySpark, saved to ADLS curated container
- Exposed as 9 SQL views in Azure Synapse Analytics — validated all 7 business questions successfully

Power BI



Executive Summary Dashboard — All 7 Business Questions at a Glance

- Interactive home page connecting to all individual Q1–Q7 detail pages
- Displays 4 critical KPIs: 9 materials below reorder, 11 RA materials, 3 expired lots, total MC provision of RM29.72K
- 183 unfulfilled production orders and 192 affected customer sales orders identified
- Each zone is a clickable navigation button linking to its full detail page